

שלב 1 - Networking

1. המשימה

בשלב הזה נכתוב client ו-server, שמתקשרים בהודעות טקסט. שימו לב שאתם כותבים את הקוד שלכם בהתאם לקונבנציות פייתון.

1.1 client

העתיקו את הקובץ client.py לפרויקט שלכם.

ממשו בקובץ זה את הפונקציה send_data:

```
def send_data(server_ip, server_port, data: bytes)
```

שמקבלת כתובת ip ו-port של שרת, ומידע (string) כלשהו. הפונקציה מתחברת לשרת בכתובת הנתונה ושולחת את המידע בפרוטוקול הבינארי הבא, באמצעות קידוד Little Endian:

message data	message size
message size <bytes>	uint32

כלומר 4 בתים שמייצגים את אורך המידע, ואז את המידע.

** לצורך הקידוד תוכלו להשתמש במודול struct של פייתון.

כדי להריץ את הקוד שכתבתם פתחו terminal והריצו

```
$ python3 client.py <message>
Sending message...
Done.
```

1.2 server

כעת צרו קובץ בשם server.py.

בקובץ זה ממשו את הפונקציה run_server:

```
def run_server(ip, port)
```

שמקבלת ip ו-port ומתחילה להאזין עליהם. כשמתקבל חיבור (connection) הפונקציה מקבלת את ההודעה מהחיבור, מדפיסה אותה, סוגרת את החיבור וממשיכה להאזין לחיבורים הבאים.

ממשו גם CLI (command-line interface) בדומה לאחד שמימשנו עבורכם ב-client.py כך שתוכלו להריץ את ה-server על ידי הרצת הפקודה הבאה:

```
$ python3 server.py 1.1.1.1 5000
Received data: hello
Received data: arazim is the best
```



1.3. בדקו את הקוד

פתחו 2 טרמינלים. הריצו server שמאזין באחד מהם, ו client שמתחבר אליו מהשני, ובדקו שהקוד שלכם אכן עובד.

הריצו את הserver שלכם על ip 127.0.0.1 בשלב הזה. זו כתובת מיוחדת, שמכונה גם כתובת loopback. כל פקטה שנשלחת לכתובת מנובת חזרה למחשב שלכם, ולא נשלחת החוצה.

1.4. דברו עם חברים

עכשיו כשיש לכם server ו client עובדים, אתם יכולים להתחבר אחד לשני!

הריצו server ממחשב אחד ו client מהשני והעבירו הודעה ביניהם.

כדי להתחבר עם הקוד שלכם בין מחשבים שונים, המחשבים יהיו צריכים להיות מחוברים לאותה הרשת (לאותו WiFi למשל), אז תוודאו שאתם אכן על אותה הרשת.¹

כאן אתם תצטרכו להשתמש ב ip'ים האמיתיים שלכם, ולא ב loopback. בשביל לבדוק מה ה-ip שלכם, הריצו את פקודת ifconfig ב terminal.

1.5. Git

אל תשכחו להעלות את הקוד שלכם ל-Git Repository שיצרתם לפרוייקט:

2. עוד דברים שאתם צריכים לדעת

2.1. Conventions

לכל אחד יש סגנון קצת שונה באיך שהוא כותב קוד, וזה בסך הכל בסדר. אבל כשאנחנו כותבים קוד, קריאות זה אחד הדברים החשובים to keep in mind. "עולם האמיתי", הקוד שאנחנו כותבים יהיה חלק ממערכת גדולה יותר, שעובדים עליה עוד אנשים חוץ מאיתנו. אנחנו נרצה שאנשים אחרים יוכלו לקרוא ולהבין את הקוד שלנו בקלות, ושלקוד שאנשים שונים כותבים יהיה סגנון יחסית אחיד.

¹ יש פה הסבר רשתות על רגל אחת לסיבה שאתם צריכים להיות על אותו Wi-Fi, והוא מיועד רק למי שזה ממש מטריד אותו. ההסבר יכול גם לבלבל, אז תרגישו חופשי לא להבין את זה כרגע ולהמשיך הלאה:

הסיבה לכך היא NAT. כתובת ה-ip שמופיעה לכם במחשב היא כתובת מקומית, והיא לא הכתובת האמיתית שהשרתים שאתם מתחברים אליהם רואים. כל המחשבים שמחוברים לראוטר מסוים בעצם מיוצגים ככתובת ip חיצונית אחת, ולראוטר יש דרך לתרגם בין הכתובות הפנימיות לכתובת החיצונית.

בגלל זה, אתם לא יכולים להתחבר עם ה client וה server שכתבתם בין מחשבים שלא נמצאים באותה רשת מקומית - כתובות ה ip שאתם רואים קיימות רק בתוכה. על אותה רשת מקומית, הפקטות שלכם לא יעברו דרך הראוטר, ולא תהיה בעיה:

ההגדרה של ה Network Adapter של המכונה כ Bridged Adapter (מה שעשינו בפרק המבוא) גם קשורה לעניין ה NAT. באופן דיפולטי, מכונות וירטואליות ב VirtualBox נמצאות בתוך רשת מקומית בתוך המחשב שלכם, כשהמחשב הוא לכאורה הראוטר. זה יוצר מצב שבו גם אם אתם והחבר שלכם על אותו ה wifi, לא תצליחו להתחבר אחד לשני דרך המכונות הוירטואליות.

הגדרת ה Network Adapter כ Bridged Adapter הופכת את המכונה ל"עוד מחשב ברשת". כלומר היא נותנת למכונה כתובת ip באותה רשת מקומית שעליה יושב המחשב האמיתי שלכם, מה שמאפשר לה לדבר ישירות עם כל המחשבים על הרשת הזאת.

בגלל זה, יש לנו קונבנציות בכתיבת קוד - אוסף הוראות לאיך הקוד צריך להראות. הקונבנציות המקובלות של פייתון הן [PEP 8](#).

כדי לוודא שהקוד שלכם עומד ב [PEP 8](#), אתם יכולים להריץ עליו [linter](#), שמחפש בו סטטית באגים ושגיאות [styling](#). דוגמאות ללינטרים לפייתון הן [flake8](#) ו [pylint](#).

שווה לציין שהרבה סביבות עבודה (כמו [PyCharm](#) למשל) בודקות שהקוד שלכם עומד ב [PEP 8](#) תוך כדי הכתיבה שלו.

■ חומר עזר

קצת מידע שיכול לעזור לכם:

- תיעוד של המודול [socket](#) של פייתון
- [מדריך](#) בסיסי לכתיבת client ו-server בסיסיים בפייתון
- בדקו מה ההבדל בין הפונקציות `socket.send` ו-`socket.sendall`.
- שימו לב שהפונקציה `socket.recv` לא מבטיחה שכל המידע יתקבל. תצטרכו להשתמש בה בלולאה.
- אם אתם נתקלים בהודעה "port is already in use" השורה הבאה אולי תעזור לכם:
`server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)`
- [שקופיות על networking - מצגת 1 עיצוב מערכות מתקדם](#)